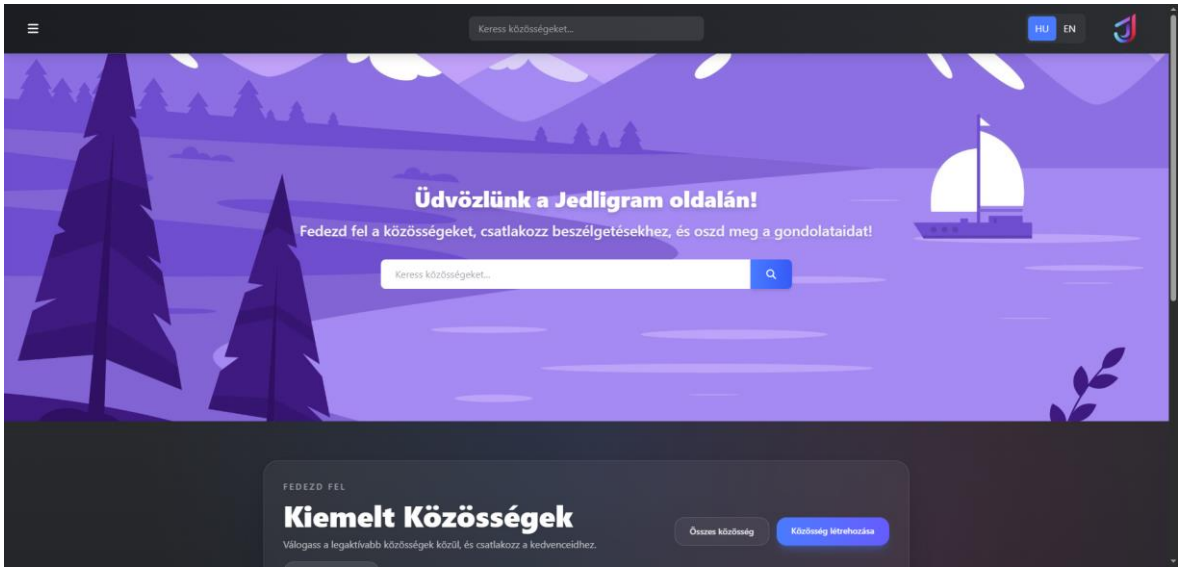
1. Áttekintés 2
2. Használt technológiák 2
3. Projektstruktúra 3
4. Futtatás és build 4
5. Routing és layoutok 4
6. Hitelesítés és 2FA 5
7. API réteg (src/api) 6
8. Közösségek (threads) 7
9. Szerepkörök és jogosultságok (Roles & Permissions) 8
10. Posztok és kommentek 9
11. Nemzetköziesítés (i18n) 9
12. Téma (light/dark) 10
13. UI frissítés egyedi eseményekkel 11
14. Tesztelés (Vitest + Cypress) 11
14.1 Komponens- és unit tesztek 11
14.2 End-to-end tesztek 12
15. Mobil futtatás (Capacitor) 13
16. Deploy (Vercel) 13
17. TypeScript interfészek (src/Interfaces) 13

1. Áttekintés

A Jedlikgram egy Reddit-szerű közösségi felület. A felhasználók közösségeket (thread-eket) böngészhetnek, posztokat olvashatnak, és bejelentkezés után posztolhatnak, kommentelhetnek és szavazhatnak.

A frontend egy React-alapú SPA (Single Page Application). A navigációt a React Router DOM kezeli, az adatforgalom axios-on keresztül a backend `/api` végpontjai felé megy.

A bejelentkezés token-alapú: sikeres belépés után a szerver egy JWT tokent küld vissza, amelyet a kliens a localStorage-ben tárol. Ezt a tokent az alkalmazás minden egyes API kérésnél automatikusan Bearer tokenként csatolja a kérés fejlécéhez, így a backend azonosítani tudja a felhasználót.



1. ábra – Főoldal (src/Pages/Home/Home.tsx)

2. Használt technológiák

A projekt fejlesztéséhez modern, széles körben alkalmazott JavaScript/TypeScript eszközöket és könyvtárakat használtunk. Az alábbiakban összefoglaljuk a főbb függőségeket, azok verzióját és szerepét.

Technológia / csomag	Verzió	Szerepe a projektben
React	19.2.0	UI komponensek, állapotkezelés
TypeScript	5.9.3	Típusbiztos fejlesztés, jobb IDE-támogatás

Technológia / csomag	Verzió	Szerepe a projektben
Vite	7.3.1	Fejlesztői szerver és produkciós build eszköz
Tailwind CSS	4.1.18	Utility-first CSS stílusozás
React Router DOM	7.13.0	Kliensoldali routing, oldal-navigáció
Axios	1.13.5	HTTP kérések a backend REST API felé
i18next	26.0.1	Többnyelvűség (internacionalizáció)
react-i18next	17.0.1	i18next React integráció
FontAwesome React	3.1.1	SVG ikonkészlet
Capacitor	8.0.2	Android natív alkalmazás csomagolás
Cypress	15.13.0	End-to-end (E2E) tesztelés böngészőben
Vitest	4.1.2	Unit és komponenstesztek
ESLint	9.39.3	Statikus kódelemzés, kódminőség-ellenőrzés

3. Projektstruktúra

A projekt mappái egyértelműen elkülönítik az alkalmazás különböző rétegeit. Ez megkönnyíti a navigálást a kódbázisban és az egyes funkciók megtalálását.

Mappa / fájl	Leírás
src/	Az alkalmazás teljes forráskódja
src/App.tsx	Route-ok és app-szintű állapot (isLoggedIn, események)
src/main.tsx	Belépési pont (BrowserRouter, i18n, téma inicializálás)
src/api/	Axios kliens és API függvények
src/Pages/	Route-onkénti oldalak (Login, Verification, Community, ...)
src/Components/	Újrahasznosítható UI komponensek
src/Layouts/	Oldallayoutok (navbar+sidebar és auth layout)
src/locales/	Fordítási fájlok (hu/en)
cypress/	End-to-end tesztek

A **Pages** mappa az oldal-szintű logikát fogja össze (adatbetöltés, navigáció, UI összeállítás), a **Components** mappa pedig a kisebb, újrahasznosítható építőelemeket.

4. Futtatás és build

A projekt Node.js + npm környezetben fut. A legfontosabb parancsok a `package.json` scripts részében vannak.

A backend elérési URL-jét és egyéb érzékeny beállításokat (például localStorage-kulcsokat) Vite környezeti változók tárolják. A szükséges változókat a `.env.example` fájl mutatja be, ezt le kell másolni `.env` névvel, és ki kell tölteni.

Hasznos parancsok:

- `npm install`
- `npm run dev`
- `npm run build`
- `npm run preview`
- `npm run lint`
- `npx vitest run`
- `npx cypress run`

5. Routing és layoutok

Az útvonalakat a `src/App.tsx` definiálja.

A **MainLayout** a legtöbb oldalt foglalja magába: tartalmaz egy Navbar-t (felső menüsáv), egy Sidebar-t (oldalsó panel) és egy Footer-t. A route-ok tartalma az Outlet komponens helyére kerül, vagyis a layout kerete változatlan marad, csak a belső tartalom cserélődik.

Az **AuthLayout** az egyszerűbb, középre igazított elrendezés, amelyet a bejelentkezési és a kétfaktoros hitelesítési (2FA) oldalak használnak.

Az alábbi képernyőkép a route definíciókat mutatja be az App.tsx fájlból:

```
<Routes>
  <Route element={<NavbarLayout isLoggedIn={isLoggedIn} />}>
    <Route index element={<Home />} />
    <Route path="search" element={<SearchResults />} />
    <Route
      path="all-communities"
      element={<AllCommunities isLoggedIn={isLoggedIn} />}
    />
    <Route
      path="create-community"
      element={<CreateCommunity isLoggedIn={isLoggedIn} />}
    />
    <Route
      path="communities/:id"
      element={<Community isLoggedIn={isLoggedIn} />}
    />
    <Route
      path="communities/:id/deleted"
      element={<DeletedCommunity />}
    />
    <Route
      path="communities/:id/posts/new"
      element={<CreatePost />}
    />
    {/* <Route path="profile" element={<Profile isLoggedIn={isLoggedIn} />} /> */}
    <Route path="users/:id" element={<UserProfile />} />
  </Route>

  <Route path="auth" element={<AuthLayout />}>
    <Route path="login" element={<LoginPage />} />
    <Route path="verification" element={<VerificationPage />} />
  </Route>
</Routes>
```

2. ábra – Route definíciók (src/App.tsx részlet)

6. Hitelesítés és 2FA

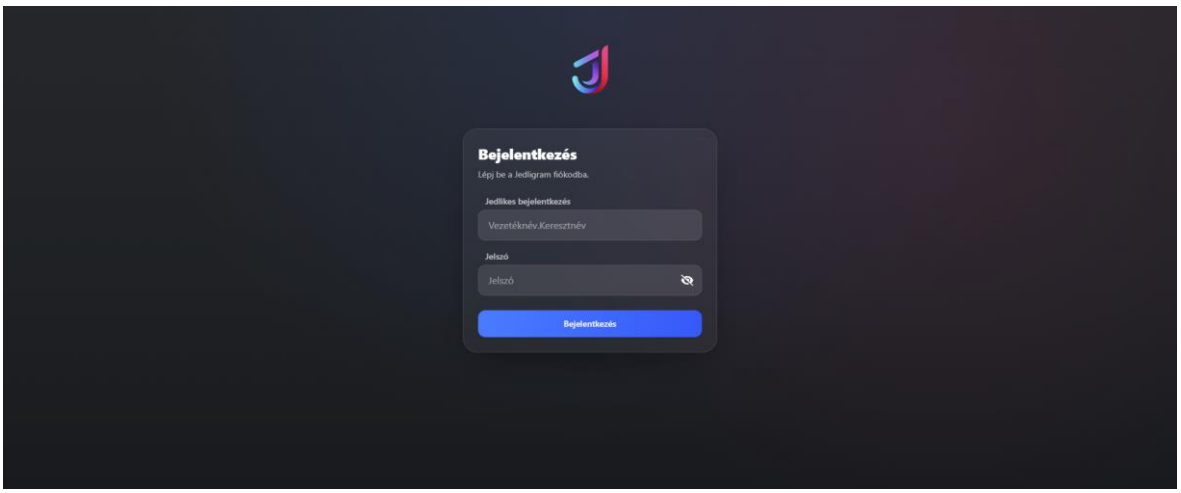
A token a localStorage-be kerül (alapértelmezett kulcs: `jedligram_token`).

A profilhoz tartozó kliensoldali adatokat a kód a `jedligram_profile` kulcs alatt tárolja. A profil kulcsneve szintén állítható `VITE_LOCAL_STORAGE_PROFILE_KEY` változóval.

A bejelentkezési UI a `/auth/login` oldalon van. Siker esetén az app `'auth-changed'` eseményt küld, így a többi komponens azonnal frissítheti a bejelentkezett állapotot.

A kétfaktoros hitelesítés (2FA) esetén a backend 202-es HTTP státusszal és egy `requires_verification` jelzéssel válaszol. Ilyenkor az alkalmazás automatikusan a

`/auth/verification` oldalra navigál, ahol a felhasználónak meg kell adnia az e-mail-jére érkezett egyszer használatos kódot.



3. ábra – Login (src/Pages/Auth/Login.tsx)

7. API réteg (src/api)

A HTTP kommunikáció központja a `src/api/httpClient.ts`, ami egy axios instance. Beállítja a baseURL-t (`VITE_BACKEND_URL`), timeoutot, és request interceptorral Bearer tokent ad a kérésekhez.

Az egyes funkcióterületekhez tartozó API hívások külön modulokban vannak: **auth.ts** (bejelentkezés, regisztráció), **threads.ts** (közösségek), **posts.ts** (posztok), **comments.ts** (kommentek), **users.ts** (felhasználók), **vote.ts** (szavazás). Ezeket az oldalak és hookok közvetlenül importálják és hívják.

```
import axios from "axios";
import { AxiosError } from "axios";

const backendUrl: string =
  import.meta.env.VITE_BACKEND_URL || "http://localhost:8000";

const authTokenName: string =
  import.meta.env.VITE_AUTH_TOKEN_NAME ?? "jedligram_token";

// Create an Axios instance
const httpClient = axios.create({
  baseURL: backendUrl,
  timeout: 10000,
});

// Request interceptor to add auth token to headers
httpClient.interceptors.request.use(
  (config) => {
    config.headers.Accept = "application/json";

    const token = localStorage.getItem(authTokenName);
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error: AxiosError) => Promise.reject(error),
);
```

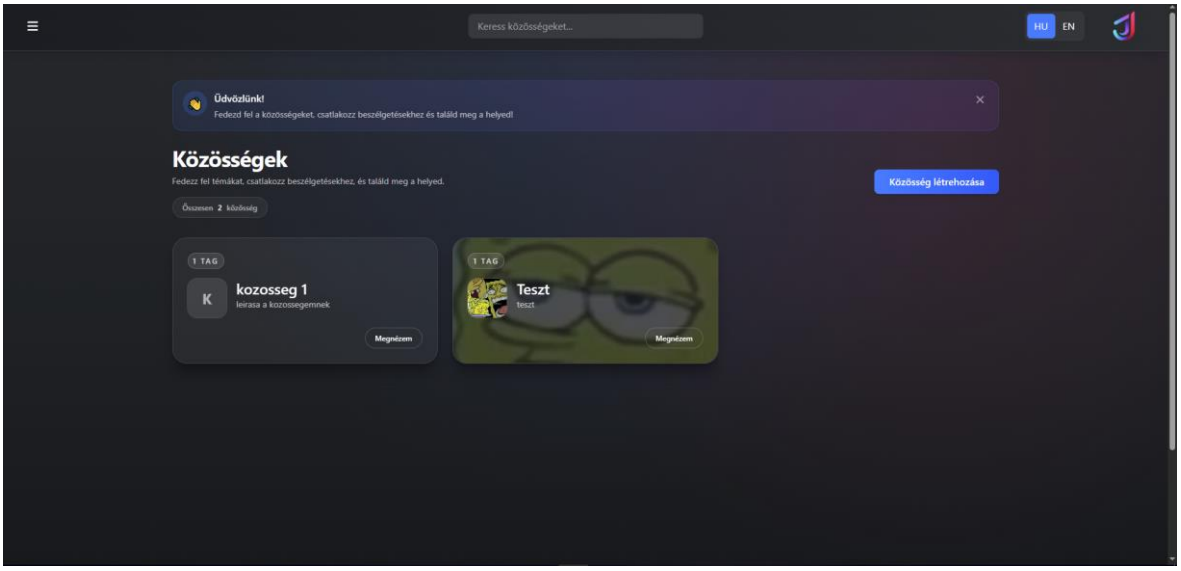
4. ábra – HTTP kliens konfigurációja (src/api/httpClient.ts)

8. Közösségek (threads)

A közösség fogalma a backendben `'thread'`. Emiatt a frontendben a közösség-kezelő API is a `threads.ts` fájlban található.

- Listaoldal (`/all-communities`): az `AllCommunities.tsx` oldal lekéri az összes elérhető thread-et a szervertől, majd CommunityCard komponensek formájában jeleníti meg azokat.
- Létrehozás (`/create-community`): a `CreateCommunity.tsx` oldal kezeli az új közösség létrehozását. Az alaplétrehozás (CreateThread hívás) után opcionálisan egy profilkép és egy fejléckép is feltölthető (UploadThreadImage / UploadThreadHeaderImage).
- Csatlakozás és kilépés: a `JoinThread` és a `LeaveThread` API függvények a `threads.ts`-ben találhatók. Ezeket a Community oldal a `useCommunity` nevű custom hookból hívja.

Megjegyzés: közösség létrehozása után a frontend nem hív külön `'join'` műveletet. A létrehozó automatikusan tag lesz. A UI oldalon a tagságot a `GetUserThreads` (felhasználó threadjei) alapján állapítja meg.



5. ábra – Közösségek (src/Pages/Communities/AllCommunities.tsx)

9. Szerepkörök és jogosultságok (Roles & Permissions)

A projekt közösség-szinten kezel jogosultságokat: a thread tagjaihoz tartozik egy `role_id` (szerepkör azonosító).

A szerepköröket az alábbi táblázat foglalja össze:

role_id	Szerepkör neve	Leírás
1	**owner (tulajdonos)**	Teljes hozzáférés: moderálhat, tagokat előléptethet vagy eltávolíthat
2	**moderator**	Posztokat moderálhat, tagokat bannolhat.
4	**banned (tiltott)**	A taglistából kiszűrve jelenik meg

A szerepkört a kliens a thread members listából olvassa ki: a Community oldal betölti a tagokat (`GetThreadMembers`), majd a saját user id alapján megkeresi a saját member rekordját és kiolvassa a `role_id`-t. Ennek alapján dönti el a felhasználói felület, hogy megjelenítse-e a moderációs lehetőségeket vagy sem.

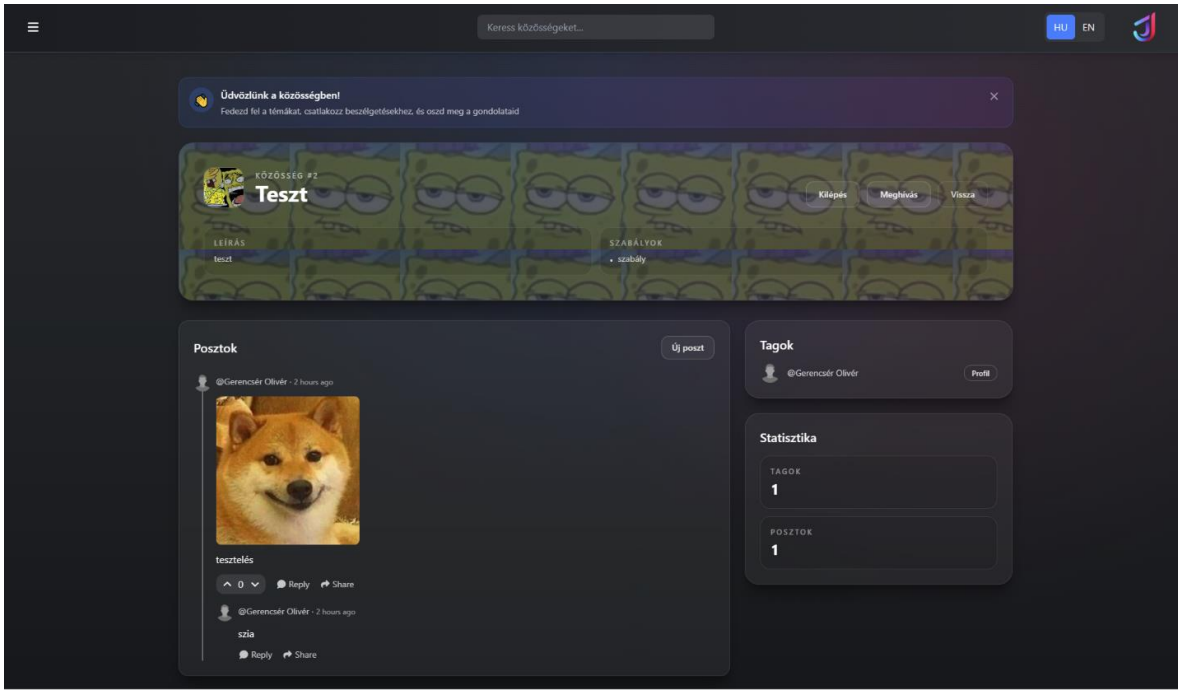
10. Posztok és kommentek

A közösség oldal (`Community.tsx`) a `PostList` komponenst használja. A PostList felel a posztok és kommentek betöltéséért, összefűzéséért és a rekurzív renderelésért.

Betöltési logika:

- Az adott thread-hez tartozó összes poszt lekérése a szervertől.
- Minden egyes poszthoz a hozzá tartozó kommentek lekérése.
- A posztok és kommentek összefűzése egy közös fa-struktúrába (`PostAndCommentData`), amely lehetővé teszi a rekurzív renderelést.
- A tartalmak rendezése: a legfrissebb elemek kerülnek felülre.

A kommentek válaszai (reply-ok) nem töltődnek be azonnal: a válaszok külön `/api/comments/{id}/replies` hívással tölthetők be.



6. ábra – Egy közösség (src/Pages/Community/Community.tsx)

11. Nemzetköziesítés (i18n)

Az alkalmazás támogatja mind a magyar, mind az angol felhasználói felületet. A fordítási rendszer az i18next + react-i18next csomagokra épül. A konfiguráció a `src/i18n.ts`-ben van.

A nyelv kiválasztása a localStorage `'language'` kulcsában tárolódik. A Navbar HU/EN gombokkal vált nyelvet.

A komponensek a `useTranslation` hookot használják (`t('kulcs')`). A fordítások a `src/locales/hu.json` és `src/locales/en.json` fájlokban vannak.

```
import i18n from 'i18next';
import { initReactI18next } from 'react-i18next';
import hu from './locales/hu.json';
import en from './locales/en.json';

const resources = {
  hu: { translation: hu },
  en: { translation: en },
};

i18n
  .use(initReactI18next)
  .init({
    resources,
    lng: localStorage.getItem('language') || 'hu',
    fallbackLng: 'hu',
    interpolation: {
      escapeValue: false,
    },
  });

export default i18n;
```

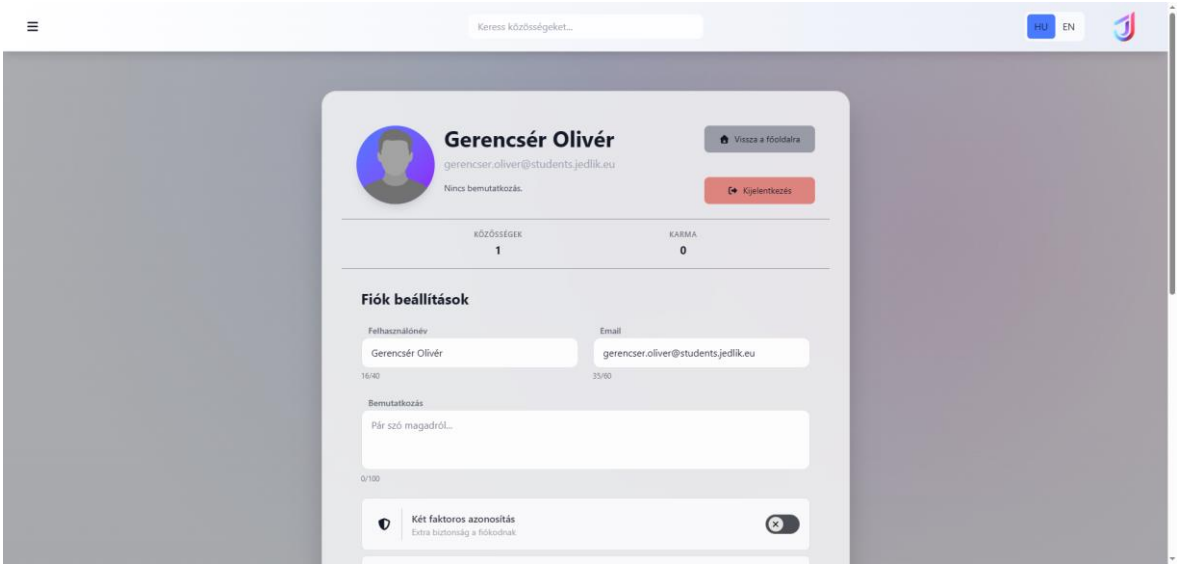
7. ábra – i18next inicializálás (src/i18n.ts)

12. Téma (light/dark)

Az alkalmazás teljes felhasználói felülete támogatja a világos (light) és a sötét (dark) megjelenési módot.

A témakezlés a `src/theme.ts` fájlban van. A választott téma a localStorage-ben `'jedligram_theme'` kulcs alatt tárolódik, így az oldal újratöltése vagy bezárása után is megmarad a beállítás.

Az `initTheme()` a `main.tsx`-ben fut, és az első render előtt beállítja a `documentElement.dataset.theme` értékét. A változás `'theme-changed'` eseményt is küld.



8. ábra – Fehér mód (profil) (src/Pages/Test/UserProfile.tsx)

13. UI frissítés egyedi eseményekkel

A projekt több helyen böngésző-eseményeket használ (`window.dispatchEvent` + `addEventListener`), hogy lazábban csatolt részek is tudjanak frissülni.

Használt események:

- `auth-changed` – be- és kijelentkezéskor
- `joined-threads-changed` – profil/joined thread lista frissülésekor
- `recent-threads-changed` – utóbb megnyitott thread-ek változásakor
- `comment-added` – új komment hatására PostList újratöltés
- `theme-changed` – témaváltáskor

14. Tesztelés (Vitest + Cypress)

14.1 Komponens- és unit tesztek

Komponens/unit teszt: Vitest + React Testing Library. Példaként az `InputComponent.test.tsx` teszt azt vizsgálja, hogy a beviteli mezőbe gépeléskor a megfelelő onChange callback hívódik-e meg a helyes értékkel.

```
import { describe, it, expect, vi } from 'vitest';
import { render, screen, fireEvent } from '@testing-library/react';
import { InputComponent } from './InputComponent';

describe('InputComponent', () => {
  it('calls onChange when typing', () => {
    const mockOnChange = vi.fn();

    render(
      <InputComponent
        label="Email"
        placeholder="Enter email"
        value=""
        onChange={mockOnChange}
      />
    );

    fireEvent.change(
      screen.getByPlaceholderText('Enter email'),
      { target: { value: 'test@test.com' } }
    );

    expect(mockOnChange).toHaveBeenCalled();
  });
});
```

9. ábra – Komponens unit teszt (src/Components/InputFields/InputComponent.test.tsx)

14.2 End-to-end tesztek

E2E teszt: Cypress – a `cypress/e2e/jedligram.cy.ts` jelenleg egy egyszerű tesztet futtat (home betölt és 'Üdvözlünk' szöveg látszik).

```
describe("Jedligram UI Tests", () => {
  beforeEach(() => {
    cy.visit("/");
  });

  it("Should load the home page and display hero section", () => {
    // Ellenőrizzük, hogy az oldal betöltött-e
    cy.get("body").should("be.visible");

    // Ellenőrizzük, hogy vannak komponensek a főoldalon
    cy.contains("Üdvözlünk").should("exist");
  });
});
```

10. ábra – E2E teszt (cypress/e2e/jedligram.cy.ts)

15. Mobil futtatás (Capacitor)

A repó tartalmaz Android projektet (`android/`), és Capacitor konfigurációt (`capacitor.config.ts`).

A web build a `dist/` mappába kerül, ezt szinkronizálja a `cap sync android`. A `package.json` tartalmaz egy `android:rebuild` scriptet, ami build + sync + run lépéseket futtat.

Az `App.tsx`-ben szerepel egy `CapacitorNavigator` komponens is, ami deep linket (`appUrlOpen`) és Android vissza gombot kezel a `@capacitor/app` segítségével.

```
import type { CapacitorConfig } from "@capacitor/cli";

const config: CapacitorConfig = {
  appId: "dev.darkiex.app",
  appName: "Jedligram",
  webDir: "dist",
};

export default config;
```

11. ábra – Capacitor konfiguráció (capacitor.config.ts)

16. Deploy (Vercel)

Az alkalmazás a Vercel felhőplatformon fut, amely automatikusan kezeli a build és a kiszolgálás folyamatát. A Vercel a GitHub repository-hoz csatlakozva minden push után automatikusan újra deployolja az alkalmazást.

A `vercel.json` rewrite szabályt tartalmaz az alkalmazáshoz: minden nem-fájl és nem `/api` útvonalat az `index.html`-re irányít, így a React Router route-ok frissítés után is működnek.

17. TypeScript interfészek (src/Interfaces)

A projekt több egyszerű interfésszel írja le a backendből érkező adatokat. Ezeket a komponensek és API függvények típusbiztonságra használják.

Főbb interfészek:

- **`ThreadData`** – egy közösség (thread) összes adata (név, leírás, kép URL, tagok száma stb.).
- **`UserData`** – egy felhasználó adatai (felhasználónév, e-mail, profilkép stb.).
- **`ResponseData`** – egy API-válasz általános adatszerkezete, amely tartalmazza a válasz státuszkódját (status) és a visszakapott adatokat (data).

Irodalomjegyzék

[1] Meta Platforms, Inc. React Documentation. Hozzáférhető: https://react.dev. Letöltve: 2025. május.

[2] Microsoft Corporation. TypeScript Language Specification és hivatalos dokumentáció. Hozzáférhető: https://www.typescriptlang.org/docs. Letöltve: 2025. május.

[3] Evan You et al. Vite – Next Generation Frontend Tooling. Dokumentáció v7.x. Hozzáférhető: https://vitejs.dev/guide. Letöltve: 2025. május.

[4] Tailwind Labs. Tailwind CSS – A utility-first CSS framework. Dokumentáció v4.x. Hozzáférhető: https://tailwindcss.com/docs. Letöltve: 2025. május.

[5] Remix Software. React Router – Declarative Routing for React v7.x. Hozzáférhető: https://reactrouter.com. Letöltve: 2025. május.

[6] axios contributors. axios – Promise based HTTP client for the browser and node.js. Hozzáférhető: https://axios-http.com/docs/intro. Letöltve: 2025. május.

[7] i18next contributors. i18next – Internationalization Framework for JavaScript. Dokumentáció v26.x. Hozzáférhető: https://www.i18next.com. Letöltve: 2025. május.

[8] Ionic, Inc. Capacitor – A cross-platform native runtime for web apps. Dokumentáció v8.x. Hozzáférhető: https://capacitorjs.com/docs. Letöltve: 2025. május.

[9] Cypress.io. Cypress – Fast, easy and reliable testing for anything that runs in a browser. Dokumentáció v15.x. Hozzáférhető: https://docs.cypress.io. Letöltve: 2025. május.

[10] Vitest contributors. Vitest – A Vite-native unit testing framework. Dokumentáció v4.x. Hozzáférhető: https://vitest.dev/guide. Letöltve: 2025. május.

[11] Vercel, Inc. Vercel Platform Documentation – Deployment, routing, and SPA configuration. Hozzáférhető: https://vercel.com/docs. Letöltve: 2025. május.